

2.6.1 กลไกการทำงานของไอพี

หน้าที่หลักของโพรโตคอลไอพีคือจัดขนาดข้อมูลให้พอเหมาะและเลือกเส้นทางที่เหมาะสมเพื่อจัดส่งคำdffแกรม ไอพีมีรูปแบบการจัดส่งคำdffแกรมเป็นแบบ “unreliable” และ “connectionless”

ความหมายของ “unreliable” คือไอพีไม่มีกลไกรับประกันว่าคำdffแกรมที่ส่งไปถึงปลายทางได้สำเร็จหรือไม่ หากมีความผิดปกติเกิดขึ้น ในระหว่างการส่งคำdffแกรมไอพีทั้งคำdffแกรมนั้นไปแล้วรายงานสาเหตุของปัญหา กลับไปด้วยโพรโตคอลไอซีเอ็มพี

ความหมายของ “connectionless” ไอพีไม่มีการกำหนดเส้นทางลำเลียงระหว่างต้นทางและปลายทาง ไอพีไม่เก็บสถานะใดๆ ของคำdffแกรมที่ส่งออกไป คำdffแกรมแต่ละชิ้น จึงเป็นอิสระจากกัน โดยมีโอกาสไปถึงปลายทางโดยไม่เรียงลำดับ

2.6.2 การประมวลคำdffแกรม

ไอพีคำdffแกรมสร้างขึ้นโดย โฮสต์ต้นทาง เมื่อถูกลำเลียงส่งไปนอกเครือข่ายอาจต้องเดินทางผ่านเราเตอร์หลายตัวจนกระทั่งถึงปลายทาง เราเตอร์และโฮสต์จะดำเนินการกับคำdffแกรมโดยกรรมวิธีเฉพาะตัว

2.6.2.1 การประมวลคำdffแกรมที่เรเตอร์

เมื่อได้รับคำdffแกรมเรเตอร์จะตรวจสอบความถูกต้องขอผลรวมตรวจสอบ จากนั้นทำการตรวจสอบเฮดเดอร์ตั้งแต่รุ่น ขนาดเฮดเดอร์ ขนาดคำdffแกรมรวม และชนิดโพรโตคอล ว่าผิดปกติหรือไม่ จากนั้นก็ลดค่า ในฟิลด์ TTL ลงหนึ่งค่า เรเตอร์ จะกำจัดคำdffแกรมทิ้ง ถ้าหากตรวจพบปัญหา ดังต่อไปนี้ ผลรวมตรวจสอบไม่ถูกต้อง พารามิเตอร์ในส่วนหัวที่ตรวจสอบผิดปกติหรือ ค่า TTL มีค่าเป็น 0 รวมทั้งกรณีที่บัฟเฟอร์ในเรเตอร์มีขนาดไม่เพียงพอ ในการจัดการกับคำdffแกรมนั้น

ถัดจากนั้นเรเตอร์ทำการตรวจสอบอปชันดังนี้คือ ออปชันกำหนดเส้นทาง ชนิดบริการ และการแบ่งแฟกเมนต์ ในขั้นนี้คำdffแกรมอาจถูกกำจัดทิ้งอีกหากการกำหนดเส้นทางในอปชัน ไม่สามารถดำเนินการได้เพราะไม่พบเรเตอร์ที่กำหนด หรือจำเป็นต้องแฟล็กแฟกเมนต์ไม่ได้อนุญาตให้ทำ

ขั้นถัดไปเป็นการปรับค่าในฟิลด์ต่างๆ ได้แก่ค่า TTL ค่าในฟิลด์อปชัน ค่าที่จำเป็นหากมี การแฟกเมนต์ และท้ายสุดเรเตอร์จะคำนวณผลรวมการตรวจสอบเพื่อใส่ในฟิลด์ผลรวมตรวจสอบก่อนที่จะนำส่งต่อไป

2.6.2.2 การประมวล คาดำแกรมที่โฮสต์ปลายทาง

เมื่อโฮสต์ปลายทางได้รับคาดำแกรมแล้วจะเริ่มตรวจสอบผลรวมตรวจสอบ จากนั้นทำการตรวจสอบถูกต้องของเฮดเดอร์ตั้งแต่ รุน ขนาดเฮดเดอร์ ขนาดคาดำแกรม และชนิดของโพรโตคอลว่าผิดปกติหรือไม่ โฮสต์จะกำจัดคาดำแกรมทิ้งหากพบว่าค่าใดค่าหนึ่งผิดปกติและแจ้งข้อผิดพลาดกลับไปด้วยไอซีเอ็มพี

หากโฮสต์พบว่าเป็นคาดำแกรมที่แฟรกเมนต์ โฮสต์ ต้องตรวจสอบว่าเป็นแฟรกเมนต์ที่อยู่ในชุดเดียวกันโดยตรวจสอบฟิลด์เลขประจำตัว แอดเดรสต้นทางและปลายทาง โพรโตคอล และตรวจสอบแฟรกเมนต์ออฟเซตว่าถูกต้องหรือไม่

โฮสต์ตั้งเวลารับคาดำแกรมให้ครบชุด ภายในเวลาในเวลาที่กำหนดตั้งแต่แฟรกเมนต์แรกเดินทางมาถึง แฟรกเมนต์ทั้งหมดถูกกำจัดทิ้งไปหากไม่สามารถรับทุกแฟรกเมนต์ในชุดนั้นภายในเวลาที่กำหนด

2.6.3 ชนิดการให้บริการ

ในรูปที่ 2-19 เป็นฟอร์แมต ของฟิลด์ TOS ซึ่งแบ่งออกเป็นสามส่วน ส่วนแรกคือ precedence ขนาด 3 บิต, ฟิลด์กำหนดคุณภาพการให้บริการ 4 บิต และฟิลด์สุดท้ายขนาด 1 บิตซึ่งไม่ได้ใช้งานและมีค่าเป็น 0

precedence ขนาด 3 บิตใช้กำหนดลำดับความสำคัญของคาดำแกรมซึ่งมีได้ 8 ระดับจาก 0 (ต่ำสุด) ถึง 7 (สูงสุด)

เราเตอร์ตรวจค่านี้อเพื่อให้บริการกับคาดำแกรมที่มีลำดับความสำคัญสูงกว่าก่อน แต่ในปัจจุบันฟิลด์นี้มักไม่ได้ใช้งาน เราเตอร์ส่วนใหญ่ไม่จัดลำดับค่าความสำคัญของคาดำแกรม



รูปที่ 2-19 ฟอร์แมตของฟิลด์ TOS

สี่บิตถัดจาก precedence ใช้กำหนดคุณภาพการให้บริการกับคาดำแกรมตามชนิดของแอปพลิเคชัน แต่ละบิตมีความหมายเฉพาะคือ D (minimize delay), T (maximize throughput), R (maximize reliability) และ C (minimize monetary cost) ดังต่อไปนี้

บิต D มีค่า “1” หมายถึงต้องการเส้นทางที่มีเวลาหน่วง (delay time) ต่ำสุดเท่าที่เรอเตอร์หาให้ได้
 บิต T มีค่า “1” หมายถึงต้องการเส้นทางที่มีความสามารถ ส่งผ่าน ข้อมูล ได้ในปริมาณมาก ๆ ใน
 หนึ่งหน่วยเวลา เรอเตอร์ต้องจัดเส้นทางที่ตรงกับความต้องการนี้

บิต R มีค่า “1” หมายถึงต้องการเส้นทางที่มีความเชื่อถือได้สูง หรือเส้นทางที่โอกาสทำให้สูญเสีย
 ค่าต่ำแกรมน้อย

บิต C มีค่า “1” หมายถึงต้องการเส้นทางที่มีค่าพารามิเตอร์ของ “cost” ต่ำ
 ทั้ง 4 บิตถ้าใดมีค่า “1” แล้วบิตที่เหลือจะมีค่าเป็น “0” เพราะเรอเตอร์ให้บริการหลายแบบพร้อมกันไม่ได้
 และหากทั้ง 4 บิตมีค่าเป็น “0” หมายถึงไม่มีบริการพิเศษใด

2.6.4 การแบ่งขนาดข้อมูล

เมื่อไอพีได้รับข้อมูลจากโปรโตคอลระดับบนและเลือกอินเทอร์เฟซ ที่ส่งข้อมูลออก ก็ทำการ
 ตรวจสอบที่ยูปรินเทอร์เฟซ หากพบว่าค่าตัวแกรมนั้นมีขนาดใหญ่เกินกว่าเอ็นทียู ไอพีจะแบ่งค่าตัวแกรมนั้น
 ออกเป็นชิ้นย่อยให้ค่าตัวแกรมนั้นมีขนาดพอเหมาะกับการเอ็นทียูวิธีนี้เรียกว่า แฟร็กเมนต์ (fragmentation) ค่าตัว
 แกรมที่ถูกแฟร็กเมนต์จะมีเฮดเดอร์ประจำตัวเอง โดยมีข้อมูลเพิ่มเติมกำหนดลักษณะของแฟร็กเมนต์
 ประกอบไปด้วย แต่ละชิ้นของแฟร็กเมนต์เป็น ค่าตัวแกรมนั้นสมบูรณ์ในตัวเอง เรอเตอร์ระหว่างทางจะไม่
 รวมค่าตัวแกรมนั้น (reassembly) แต่จะปล่อยให้เป็นหน้าที่ของเรอเตอร์ปลายทาง การแฟร็กเมนต์ค่าตัวแกรมนั้น
 ของไอพีใช้ฟิลด์ 3 ฟิลด์กำหนดข่าวสารเกี่ยวกับการแฟร็กเมนต์ดังต่อไปนี้

identification ขนาด 16 บิต : หมายเลขประจำชิ้นค่าตัวแกรมนั้น หากต้องการแฟร็กเมนต์ก็ใช้เป็น
 ค่าที่บอกว่าเป็นค่าตัวแกรมนั้นชุดเดียวกัน สถานที่สร้างค่าตัวแกรมนั้นจะเพิ่มค่านี้นี้ครั้งละหนึ่งให้แต่ละค่าตัวแกรมนั้น
 แต่ละชุดที่ส่งออกไป

flag ขนาด 3 บิต : ใช้ควบคุมและแสดงรูปแบบแฟร็กเมนต์ แบ่งออกเป็น 3 บิต

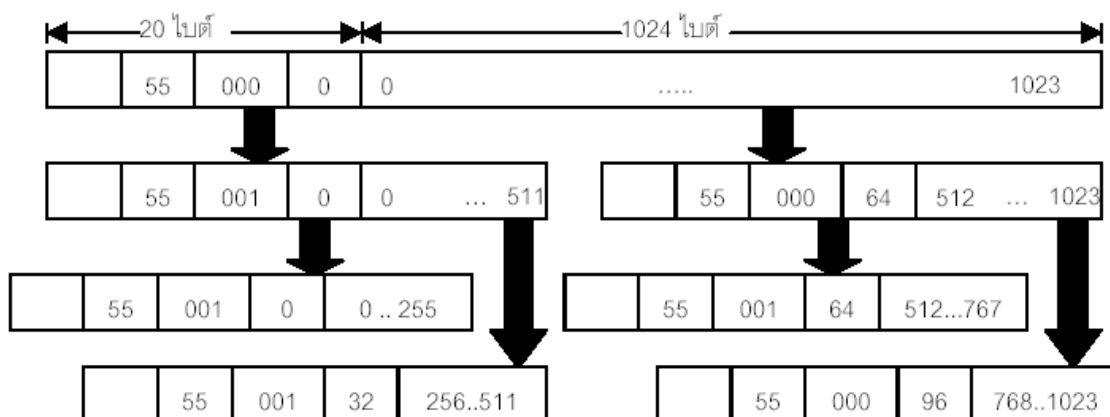
บิตแรก ไม่ได้ใช้งานและมีค่าเป็น 0 เสมอ

บิต D ใช้กำหนดว่า ให้มีแฟร็กเมนต์ได้หรือไม่ หากมีค่า “0” หมายถึงให้เรอเตอร์แฟร็ก
 เมนต์ได้ตามความจำเป็น หากมีค่าเป็น “1” หมายถึงบังคับไม่ให้แฟร็กเมนต์ ถ้าเรอเตอร์ระหว่างทางจำเป็น
 ต้องแฟร็กเมนต์แต่ถูกบังคับไม่ให้แฟร็กเมนต์ด้วยบิตนี้ เรอเตอร์ทำการทิ้งค่าตัวแกรมนั้นไปและแจ้งความ
 ผิดพลาดกลับไปยังต้นทาง

บิต M หากเป็น “0” หมายถึง เป็นค่าตัวแกรมนั้นแฟร็กเมนต์สุดท้าย หากเป็น “1” หมายถึง
 ยังมีแฟร็กเมนต์ตามมาอีก

fragment offset ขนาด 13 บิต : เรอเตอร์ทำการแฟร็กเมนต์ข้อมูลออกเป็นบล็อกแต่ละบล็อกต้อง
 เป็นจำนวนเท่าของ 8 ไบต์ ฟิลด์นี้ใช้บอกตำแหน่งออฟเซตของข้อมูลในแต่ละแฟร็กเมนต์อ้างอิงข้อมูล

2.6.4.1 การแฟรกเมนต์



รูปที่ 2-20 การแฟรกเมนต์

รูปที่ 2-20 สาธิตการแฟรกเมนต์ดาต้าแกรมโดยสมมติให้ดาต้าแกรมขนาด 1044 ไบต์ ซึ่งประกอบด้วยเฮดเดอร์ 20 ไบต์ และข้อมูล 1024 ไบต์ ดาต้าแกรมนี้เดินทางผ่าน 2 เครือข่ายคือ N1 และ N2 ซึ่งมีเอ็นทียูเท่ากับ 532 และ 276 ตามลำดับ ค่าที่ใช้เป็นเพียงค่าสมมติเพื่อให้เข้าใจการทำงานโดยง่าย

กำหนดให้ไม่มีการใช้ออปชัน ไอพีเฮดเดอร์จะมีขนาดคงที่ 20 ไบต์ เครือข่าย N1 มีเอ็นทียู 532 ไบต์จะแฟรกเมนต์ดาต้าแกรมหากข้อมูลมีขนาดเกิน 512 ไบต์ และเครือข่าย N2 มีเอ็นทียู 276 ไบต์ จะแฟรกเมนต์ดาต้าแกรมหากข้อมูลมีขนาดเกิน 256 ไบต์

เมื่อดาต้าแกรมขนาด 1044 ไบต์ ถูกส่งผ่านเราเตอร์ R1 ไปยัง N1 เราเตอร์ R1 ทำการแบ่งดาต้าแกรมออกเป็น 2 แฟรกเมนต์ๆ ละ 532 ไบต์ คือไอพีเฮดเดอร์ 20 ไบต์ และข้อมูล 512 ไบต์ ในฟิลด์ identification มีค่าเท่ากับ 55 โดยที่บิต M ของแฟรกเมนต์แรกมีค่าเป็น “1” เพื่อบอกว่ามีแฟรกเมนต์อื่นตามมา และบิต M แฟรกเมนต์ที่สองมีค่าเป็น “0” เพื่อบอกว่าเป็นแฟรกเมนต์สุดท้าย ส่วนค่าออฟเซตของแฟรกเมนต์แรกมีค่าเป็น 0 และออฟเซตของแฟรกเมนต์ที่สองมีค่าเป็น 64 (มาจาก 512/8) เมื่อแต่ละแฟรกเมนต์ผ่าน R2 เข้าสู่เครือข่าย N2 แต่ละแฟรกเมนต์ถูกแบ่งออกเป็นอีก 2 แฟรกเมนต์ทำให้มีแฟรกเมนต์หลังจากผ่าน N2 เป็นจำนวน 4 แฟรกเมนต์ด้วยวิธีเดียวกัน

2.6.4.2 การรีแอสเซมบลี

สถานีปลายทางทำหน้าที่ รีแอสเซมบลี (reassembly) หรือรวมแต่ละแฟรกเมนต์เข้าด้วยกัน แฟรกเมนต์ที่อยู่ในดาต้าแกรมชุดเดียวกันต้องมีค่าต่อไปนี้ตรงกัน คือ identification, แอดเดรสต้นทาง, แอดเดรสปลายทาง และชนิดโปรโตคอล

ค่า total length ในแต่ละแฟรกเมนต์ของดาต้าแกรมบอกเพียงขนาดดาต้าแกรมนั้นหากแต่ละดาต้าแกรมมาถึงปลายทางโดยไม่เรียงลำดับ สถานีปลายทางไม่มีโอกาสทราบล่วงหน้าได้ว่า ดาต้าแกรมทั้งชุดมีขนาดเท่าใด ด้วยเหตุนี้สถานีปลายทางจำเป็นต้องเตรียมบัฟเฟอร์เพื่อการรีแอสเซมบลีให้เพียงพอ

2.6.4.3 ข้อคำนึงในการแฟรกเมนต์

การทำ แฟรกเมนต์ไม่ได้เกิดเฉพาะกรณีที่ต้องส่งคำสั่งแกรมข้ามเครือข่ายเท่านั้น แม้แต่ในเครือข่ายเดียวกันก็ อาจจำเป็นต้องแฟรกเมนต์คำสั่งแกรมด้วยเช่น โพรโตคอลเอ็นเอฟเอส มักส่งข้อมูลครั้งละ 8,192 ไบต์ ซึ่งใหญ่กว่าเอ็นทียูในเครือข่ายแลนทั่วไป

หากสถานีปลายทางไม่สามารถรับทุกแฟรกเมนต์ได้ครบด้วยสาเหตุใดก็ตามต้องทิ้งคำสั่งแกรมทั้งหมดไป ในกรณีที่ของเอ็นเอฟเอสนั้นสถานีต้นทางต้องส่งคำสั่งแกรมทั้ง 8,192 ไบต์ที่ต้องทำการแฟรกเมนต์ซ้ำเช่นเดิมอีก ทำให้เพิ่มภาระในระบบเครือข่ายมากขึ้น ในเครือข่ายที่มีความผิดปกติและต้องมีการส่งแฟรกเมนต์ซ้ำจะสร้างภาระให้เครือข่ายสูงมาก

2.7 แผนภูมิสถานะที่ซีพี

กระบวนการทำงานของทีซีพีซึ่งประกอบด้วยการเริ่มต้นการเชื่อมต่อ การขนถ่ายข้อมูล และการปิดการเชื่อมต่อ แต่ละขั้นตอนในกระบวนการเหล่านี้มีสถานะกำกับเพื่อกำหนดแบบแผนการทำงาน การเปลี่ยนจากสถานะหนึ่งไปสู่อีกสถานะหนึ่งขึ้นอยู่กับรูปแบบเซกเมนต์ที่ได้รับ ทีซีพีทั้งสองด้านต้องเก็บรักษาสถานะการเชื่อมโยงแต่ละส่วนไว้ตลอดเวลา

2.7.1 สถานะที่ซีพี

โพรโตคอลชนิด “Connection oriented” อย่างเช่นทีซีพีมีสถานะ กำกับการทำงานแต่ละขั้นตอนสถานะของทีซีพีช่วยให้โพรโตคอลมีแบบแผนการทำงานที่ ชัดเจนและสามารถรองรับการทำงานได้หลายรูปแบบ เพื่อความเข้าใจเกี่ยวกับสถานะเหล่านี้ขอให้พิจารณาถึงแผนภูมิในรูปที่ 4-1 ประกอบคำอธิบาย แผนภูมินี้แสดงเฉพาะการเปลี่ยนแปลงสถานะตามเหตุการณ์ปกติที่เกิดขึ้นทั้งด้านทีซีพีไคลเอ็นต์และเซิร์ฟเวอร์โดยสรุปรวมไว้ในรูปเดียวกัน

2.7.1.1 การเริ่มต้นการเชื่อมต่อ

หากแยกพิจารณาสถานะทีซีพีไคลเอ็นต์ และ เซิร์ฟเวอร์ออกจากกันตามขั้นตอนการเริ่มต้นการเชื่อมต่อเพื่อจัดหมวดหมู่ให้เข้าใจง่าย สถานะของเซิร์ฟเวอร์และไคลเอ็นต์เมื่อเริ่มต้นการเชื่อมต่อ เป็นดังนี้

2.7.1.1.1 สถานะเซิร์ฟเวอร์ขั้นตอนการเริ่มต้นการเชื่อมต่อ

CLOSED สถานะจำลองที่สร้างขึ้นเพื่ออ้างอิง ในสถานะนี้ไม่มีการจัดสรรหน่วยความจำเพื่อเก็บค่าใด

LISTEN เซิร์ฟเวอร์รอคอยการขอบริการจากไคลเอ็นต์ หรือเรียกว่า เซิร์ฟเวอร์เปิดการเชื่อมต่อแบบพาสซีฟ

SYN_RECEIVED เซอร์ฟเวอร์ได้รับ SYN และส่ง SYN/ACK และรอคำตอบจากไคลเอนต์(รอ ACK)

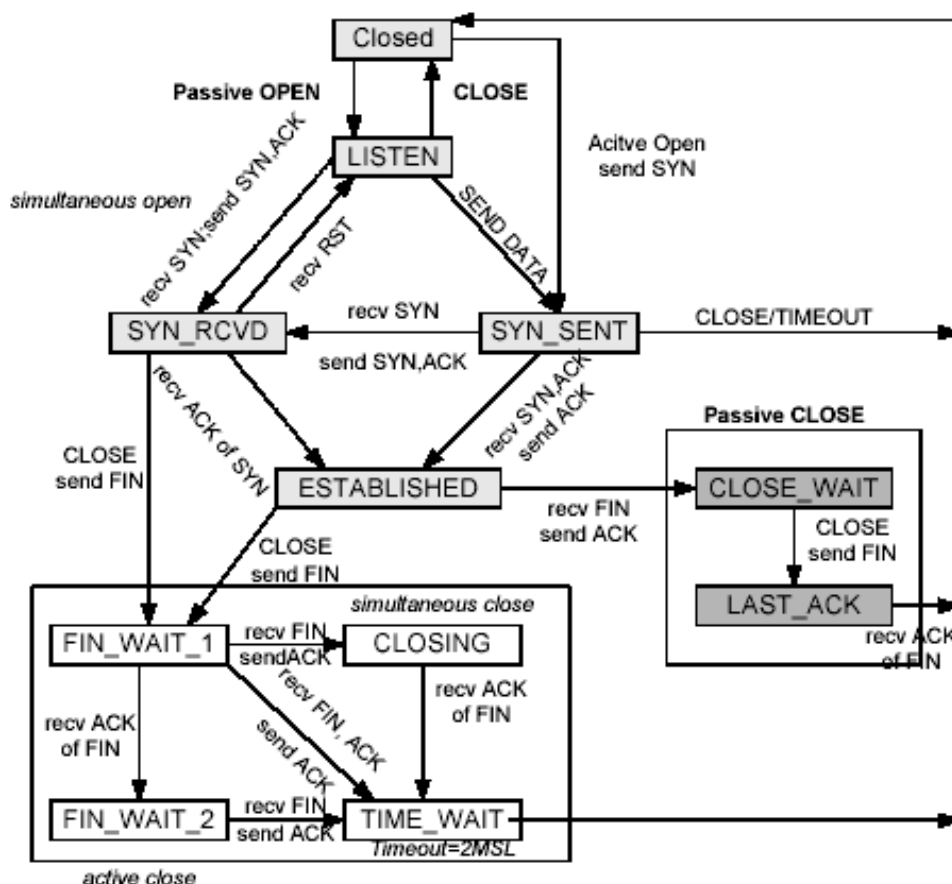
ESTABLISHED เซอร์ฟเวอร์ได้รับ ACK การสถาปนาด้านเซอร์ฟเวอร์เสร็จสมบูรณ์และพร้อมโอนถ่ายข้อมูล

2.7.1.1.2 สถานะไคลเอนต์ขั้นตอนการเริ่มต้นการเชื่อมต่อ

CLOSED สถานะจำลองที่สร้างขึ้นเพื่ออ้างอิง ในสถานะนี้ไม่มีการจัดสรรหน่วยความจำเพื่อเก็บค่าใด

SYN_SENT ไคลเอนต์ส่ง SYN และรอคอยการจับคู่เชื่อมต่อ (ตอบรับ) โดยเรียกว่าไคลเอนต์ขอเปิดการเชื่อมต่อแบบแอกทีฟ

ESTABLISHED ไคลเอนต์ได้รับ SYN/ACK จากเซอร์ฟเวอร์ และตอบกลับไปด้วย ACK การเริ่มต้นการเชื่อมต่อด้านไคลเอนต์เสร็จสมบูรณ์และพร้อมถ่ายโอนข้อมูล



รูปที่ 2-21 แผนภูมิสถานะทีซีพี

2.7.1.2 การปิดการเชื่อมต่อ

เมื่อเริ่มต้น การเชื่อมต่อ เสร็จสิ้นแล้ว ทั้งไคลเอ็นต์ และเซิร์ฟเวอร์ จะอยู่ในสถานะ ESTABLISHED จนกระทั่งฝ่ายหนึ่งไม่มีข้อมูลส่งและขอปิดการเชื่อมต่อโดยส่ง เชกเมนต์ FIN การปิดการเชื่อมต่อแต่ละฝ่ายจะผ่านสถานะต่อไปนี้เป็นลำดับ

2.7.1.2.1 สถานะฝ่ายที่ขอปิดการเชื่อมต่อ

FIN_WAIT_1 ส่ง FIN และเข้าสู่สถานะรอคอยเพื่อรอการตอบยืนยันว่าได้รับคำสั่งปิดการเชื่อมต่อ ในขั้นนี้อาจยังมีข้อมูลส่งมาจากอีกฝ่ายหนึ่งได้

FIN_WAIT_2 ฝ่ายขอปิดได้รับ ACK ยืนยันว่าอีกฝ่ายได้รับเชกเมนต์ขอปิดแล้ว (แต่อีกฝ่ายอาจไม่พร้อมที่จะปิดเพราะยังมีข้อมูลที่ต้องส่งอยู่)

CLOSING ฝ่ายขอปิดได้รับเชกเมนต์ FIN และจะส่ง ACK ตอบกลับ

TIME_WAIT สถานะรอคอยเพื่อรอการปิดการเชื่อมต่อ

CLOSED ปิดการเชื่อมต่อแล้ว ข้อมูลการเชื่อมต่อถูกกำจัดทิ้งหมด

2.7.1.2.2 สถานะฝ่ายได้รับคำร้องขอปิดการเชื่อมต่อ

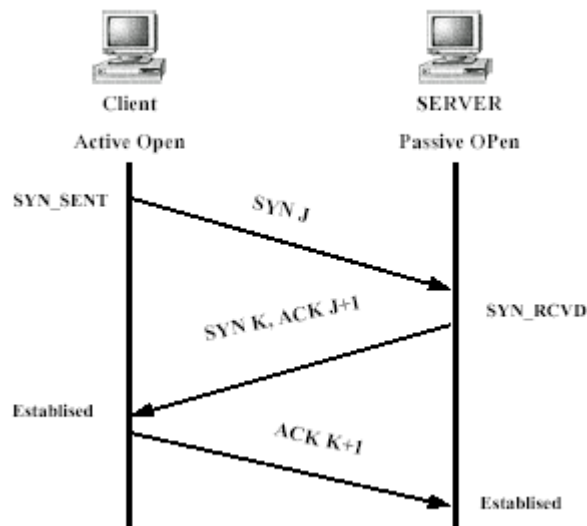
CLOSED_WAIT ได้รับ FIN ขอปิดการเชื่อมต่อ ที่ซีพีส่ง ACK หากโปรแกรมประยุกต์ยังมีข้อมูลต้องนำส่งอยู่ ที่ซีพีจะรอนกว่าโปรแกรมประยุกต์ส่งปิด

LAST_ACK โปรแกรมประยุกต์ส่งปิดการส่งข้อมูล ที่ซีพีส่ง FIN และคอยการตอบรับหากได้รับ ACK เมื่อใดก็จะเข้าสู่สถานะ CLOSED

CLOSED ปิดการเชื่อมต่อแล้ว ข้อมูลการเชื่อมโยงถูกกำจัดทิ้งหมด

2.7.1.3 การรับส่ง เชกเมนต์ในขั้นตอนการเริ่มต้นการเชื่อมต่อ

รูปที่ 2-22 แสดงขั้นตอนเริ่มต้นการเชื่อมต่อระหว่างไคลเอ็นต์และเซิร์ฟเวอร์คู่หนึ่ง โดยสมมติให้ไคลเอ็นต์เปิดการเชื่อมต่อแบบแอกทีฟและเซิร์ฟเวอร์เปิดการเชื่อมต่อแบบพาสซีฟไคลเอ็นต์เริ่มต้นจากสถานะ CLOSED และเข้าสู่สถานะ SYN_SENT หลังจากส่ง เชกเมนต์ SYN เซิร์ฟเวอร์เมื่อได้ รับเชกเมนต์ SYN ก็จะตอบกลับด้วยเชกเมนต์ SYN/ACK และเปลี่ยนจากสถานะ LISTEN เข้าสู่สถานะ SYN_RCVD ไคลเอ็นต์เมื่อได้รับเชกเมนต์นี้จะส่ง เชกเมนต์ ACK กลับไปให้ เซิร์ฟเวอร์และเข้าสู่สถานะ ESTABLISHED เมื่อเซิร์ฟเวอร์ได้รับ เชกเมนต์นี้ ก็จะเข้าสู่สถานะ ESTABLISHED



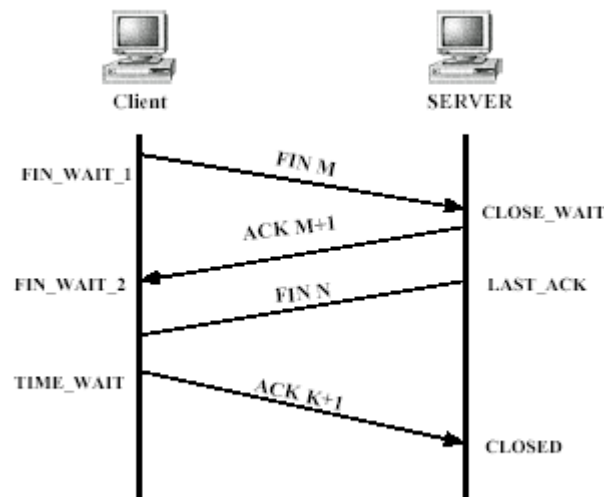
รูปที่ 2-22 สถานะการเริ่มต้นการเชื่อมต่อที่ซีพี

2.7.1.4 การรับส่ง เซกเมนต์ในขั้นตอนการปิดการเชื่อมต่อ

หากย้อนไปพิจารณารูปที่ 2-21 จะพบว่าไคลเอ็นต์และเซิร์ฟเวอร์เป็นฝ่ายเริ่มปิดการเชื่อมต่อได้ทั้งสิ้น ในรูปที่ 2-23 จะแสดงเฉพาะกรณีไคลเอ็นต์เป็นผู้ร้องขอการปิดการเชื่อมต่อ ดังนั้นไคลเอ็นต์จะปิดแบบแอกทีฟและเซิร์ฟเวอร์จะปิดแบบพาสซีฟ

ขั้นตอนเริ่มต้นเมื่อไคลเอ็นต์ไม่มีข้อมูลจัดส่งอีกต่อไปแล้วก็แจ้งให้ เซิร์ฟเวอร์ทราบโดยส่งเซกเมนต์ FIN และเข้าสู่สถานะ FIN_WAIT_1 เมื่อเซิร์ฟเวอร์ได้รับก็จะตอบกลับด้วยเซกเมนต์ ACK และเข้าสู่สถานะ CLOSE_WAIT เมื่อไคลเอ็นต์ได้รับเซกเมนต์นี้จะเปลี่ยนจากสถานะ FIN_WAIT_1 ไปสู่ FIN_WAIT_2

ในจังหวะนี้เซิร์ฟเวอร์อาจยังมีข้อมูลที่น่าส่งไปยังไคลเอ็นต์ได้อยู่ เมื่อเซิร์ฟเวอร์ส่งข้อมูลหมดสิ้นแล้วก็ส่งเซกเมนต์ FIN แจ้งให้ไคลเอ็นต์ทราบและเปลี่ยนสถานะไปเป็น LAST_ACK ด้านไคลเอ็นต์เมื่อได้รับเซกเมนต์ FIN ก็รับทราบว่าเซิร์ฟเวอร์พร้อมปิดตัวเองแล้ว ไคลเอ็นต์ตอบกลับด้วยเซกเมนต์ ACK และเข้าสู่สถานะ TIME_WAIT เพื่อปิดตัวเองภายในเวลา 240 วินาที ในขณะที่เซิร์ฟเวอร์ปิดการเชื่อมต่อโดยเข้าสู่สถานะ CLOSED เมื่อได้รับเซกเมนต์ ACK นี้



รูปที่ 2-23 สถานะการปิดการเชื่อมต่อ

สถานะ TIME_WAIT เป็นสถานะรอคอยเพื่อเข้าสู่การปิดการเชื่อมต่อ ข้อสังเกตประการเชื่อมต่อหนึ่งนั้นไม่สามารถใช้ได้จนกว่าจะพ้นช่วงเวลานี้ไป หากมี เซกเมนต์ใดๆ ที่เพิ่งมาถึงในช่วงเวลานี้ (เนื่องจากอาจถ่วงเวลาอยู่) และมี ข้อสังเกตที่อยู่ในสถานะรอคอย เซกเมนต์นั้นจะถูกกำจัดทิ้ง ระยะเวลาในสถานะนี้มีค่าเป็นสองเท่าของค่าช่วงชีวิตนานที่สุดของเซกเมนต์จึงมักเรียกสถานะนี้ว่า “2MSL Wait state” ค่าช่วงชีวิตนานที่สุดของเซกเมนต์ที่กำหนดใน RFC 793 คือ 120 นาที ดังนั้น 2MSL จึงมีค่า 240 วินาที (ในบางระบบปฏิบัติการอาจใช้ค่า MSL เพียง 30 วินาที หรือ 1 นาที)

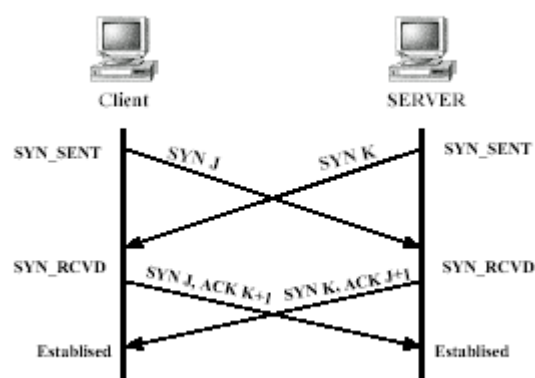
2.7.2 รูปแบบการเปิดและปิดการเชื่อมต่ออื่น

การเปิดและปิดการเชื่อมต่อที่ อธิบายก่อนหน้านี้เป็นรูปแบบที่เกิดขึ้นโดยปกติระหว่างไคลเอ็นต์และเซิร์ฟเวอร์ แต่ไคลเอ็นต์ และเซิร์ฟเวอร์อาจเข้าสู่สถานการณ์บางรูปแบบตามรูปที่ 2-21 คือ การเปิดพร้อมกัน (Simultaneous open) และ การปิดพร้อมกัน (Simultaneous close) นอกจากนี้ยังมีลักษณะการเชื่อมโยงอีก 2 แบบคือ การเปิดครึ่งฝ่าย (half-open) และ การปิดครึ่งฝ่าย (half-close)

2.7.2.1 การเปิดพร้อมกัน

การเปิดพร้อมกันเกิดขึ้นในกรณีที่สถานีคู่หนึ่งขอเปิดการเชื่อมต่อแบบแอกทีฟพร้อมกันในขณะเวลาเดียวกัน แต่ละฝ่ายส่ง เซกเมนต์ SYN โดยกำหนดพอร์ตต้นทางและปลายทางที่สมนัยกัน ตัวอย่างเช่น สถานี A มีพอร์ตต้นทางเท่ากับ 1000 ขอเปิดไปยังพอร์ต 2000 ของสถานี B ในขณะเดียวกันที่สถานี B ใช้พอร์ตต้นทาง 2000 ขอเปิดพอร์ต 1000 ของสถานี A

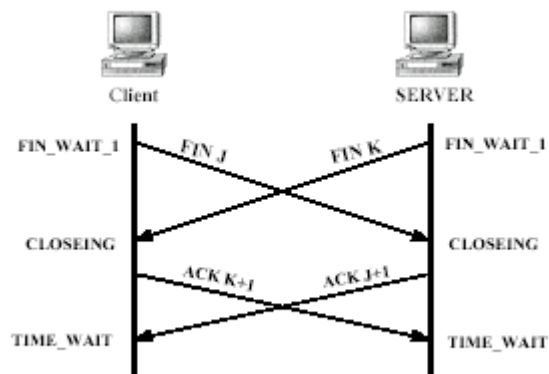
การเปิดพร้อมกันมี เซกเมนต์แลกเปลี่ยนจำนวน 4 เซกเมนต์ดังรูปที่ 2-24 โดยไม่มีการกำหนดว่าฝ่ายใดเป็นไคลเอนต์หรือเซิร์ฟเวอร์ทั้งสองด้านเปิดการเชื่อมต่อแบบพาสซีฟ ระหว่างกันและมีการเชื่อมโยงทางที่ซีพีสองส่วน ในขณะที่การเปิดพร้อมกันจะมีการเชื่อมโยงทางที่ซีพีส่วนเดียว



รูปที่ 2-24 การเปิดพร้อมกัน

2.7.2.2 การปิดพร้อมกัน

การปิดการเชื่อมต่อตามปกติเริ่มต้นจากฝ่ายหนึ่งซึ่งอาจเป็นไคลเอนต์หรือเซิร์ฟเวอร์ขอปิดการเชื่อมต่อแบบแอกทีฟโดยส่ง เซกเมนต์ FIN ไปยังอีกฝ่ายหนึ่ง แต่หากทั้งสองฝ่ายต่างขอปิดแบบแอกทีฟพร้อมกันจึงเรียกว่าการปิดพร้อมกัน



รูปที่ 2-25 การปิดพร้อมกัน

รูปที่ 2-25 แสดงการปิดพร้อมกัน เริ่มจากทั้งสองฝ่ายอยู่ในสถานะ ESTABLISHED และต่างส่งเซกเมนต์ FIN ให้อีกฝ่ายหนึ่ง และเข้าสู่สถานะ FIN_WAIT_1 เช่นกัน จำนวนเซกเมนต์ที่ใช้ในการปิดพร้อมกันยังคงเท่ากับการปิดตามปกติ แต่ลำดับการได้ตอบเซกเมนต์จะแตกต่างกัน

2.7.2.3 การเปิดครึ่งฝ่าย (half open)

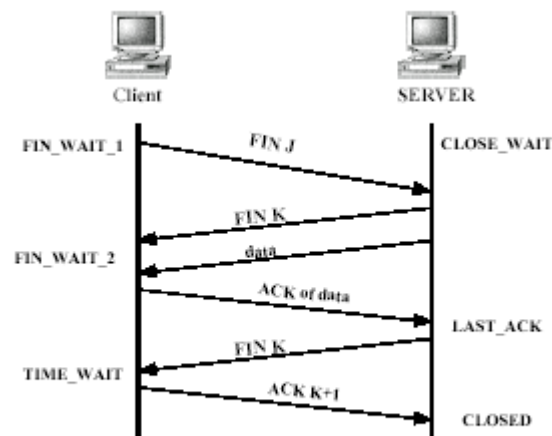
หากทีซีพีฝ่ายใดฝ่ายหนึ่งหยุดการทำงานโดยอีกฝ่ายหนึ่งไม่ทราบ ทีซีพีอีกฝ่ายที่ยังทำงานแบบเปิดครึ่งฝ่าย (half-open) โดยปกติแล้วกรณีเช่นนี้มักเกิดจากโฮสต์ฝ่ายหนึ่งเกิดปัญหากระทั่งต้องปิดเครื่อง ในระหว่างนั้นหากไม่มีหารส่งข้อมูลใด อีกฝ่ายหนึ่งจะยังคงอยู่ในสถานะ ESTABLISHED และรอคอยการรับส่งข้อมูลอยู่ ตัวอย่างเช่น โคลเอ็นต์ขอใช้บริการเทเลเน็ตจากเซิร์ฟเวอร์ หากเซิร์ฟเวอร์หยุดการทำงานและต้องรีบูตเครื่องใหม่ สถานภาพที่เชื่อมโยงกับโคลเอ็นต์จะสูญหายไปหมดจากการรีบูตโดยโคลเอ็นต์ไม่ทราบ โคลเอ็นต์จึงเปิดการทำงานเพียงครึ่งเดียว

เมื่อโคลเอ็นต์ ส่ง เซกเมนต์ไปให้เซิร์ฟเวอร์ เซกเมนต์นั้นจะไม่มี ช็อกเก็ตใดรองรับกล่าวอีกนัยหนึ่งคือเซิร์ฟเวอร์ไม่สามารถแยกแยะได้ว่าเซกเมนต์นั้นเป็นของการเชื่อมโยงใด สิ่งที่เซิร์ฟเวอร์ดำเนินการก็คือส่งเซกเมนต์รีเซ็ต โดยเซตแพ็ก RST เพื่อแจ้งให้โคลเอ็นต์ ยกเลิกการเชื่อมต่อนั้น

2.7.2.4 การปิดครึ่งฝ่าย (half close)

เนื่องจากการเชื่อมโยงทีซีพี เป็นแบบฟูลดูเพล็กซ์ ข้อมูลสามารถเดินทางได้ระหว่างโคลเอ็นต์และเซิร์ฟเวอร์ทั้งสองทิศทางอย่างเป็นอิสระ แต่ละฝ่ายจึงสามารถปิดตัวเองลงเมื่อไม่มีข้อมูลใดต้องส่งอีกต่อไป โดยยังสามารถรับข้อมูลจากอีกฝ่ายหนึ่งได้ การเชื่อมโยงลักษณะนี้เรียกว่า ปิดครึ่งฝ่าย (halfclose)

ตัวอย่างในรูปที่ 2-26 ไคลเอนต์ขอปิดการเชื่อมต่อโดยส่งเซกเมนต์ FIN และเข้าสู่สถานะ FIN_WAIT_1 เมื่อเซิร์ฟเวอร์ได้รับเซกเมนต์ FIN และตอบรับ (เซกเมนต์ ACK J+1) แล้วก็จะเข้าสู่สถานะ CLOSE_WAIT และเมื่อไคลเอนต์ได้รับเซกเมนต์ตอบรับก็จะเข้าสู่สถานะ FIN_WAIT_2 และเตรียมปิดการเชื่อมต่อ ในจังหวะนี้แอปพลิเคชันด้านเซิร์ฟเวอร์อาจยังมีข้อมูลที่ต้องส่งจึงยังไม่ปิดการเชื่อมต่อ ไคลเอนต์ปิดการส่งแต่ยังคงต้องรับข้อมูลจากเซิร์ฟเวอร์อยู่



รูปที่ 2-26 การปิดครึ่งเดียว (half close)

ขั้นตอนถัดไปตามรูปที่ 2-26 สมมติว่าเซิร์ฟเวอร์มีข้อมูลต้องส่งเพียงเซกเมนต์เดียว (เซกเมนต์ data) เมื่อเซิร์ฟเวอร์ได้รับเซกเมนต์ตอบรับ (เซกเมนต์ ACK of data) ก็สั่งปิดการเชื่อมโยงโดยส่งเซกเมนต์ FIN และเข้าสู่สถานะ LAST_ACK ถัดจากนั้นการปิดการเชื่อมโยงก็จะดำเนินต่อไปตามปกติ